

程序设计 II 第 15 周 题目汇总（完整版·模板）

来源：Matrix·课程「程序设计 II（25 级 曾坤 计算机学院）」含：完整题面、提交文件、平台提供的初始文件（.hpp/main.cpp 等）。选择题含标准答案。

共 7 项：课后 1₄、课堂 1₂、理论题（20 题）

周 15-课后 1 区间管理器（编程题）

题面

设计并实现类模板 `IntervalManager<typename T, int Limit>`，内部最大存储容量 `Limit` 由非类型模板参数编译期确定。核心业务：

1. **带交集合并的插入（ADD）**：插入新区间 `[start, end]` 时遍历所有已存区间，若与某些旧区间有交集或边缘恰好闭合重合，则融合成一个更大区间（新边界为所有有交集区间的最小起点和最大终点）。
2. **容量超限的最短淘汰**：合并完成后若区间总数超过 `Limit`，淘汰长度 `(end - start)` 最短的区间（多项相同则淘汰 `start` 更小者）。
3. **查询与排序打印（QUERY）**：按 `start` 升序打印所有区间。

输入：每行一条指令，`INT s e`（向 `int` 管理器插入）或 `DOUBLE s e`（向 `double` 管理器插入）。**输出**：每条指令输出一行，按起点升序，区间用空格隔开（浮点保留 1 位小数），形如 `[10,20] [30,40]`。

样例输入：

```
INT 1 5
INT 4 10
INT 15 20
INT 8 17
INT 25 30
INT 28 35
INT 0 3
```

样例输出：

```
[1,5]
[1,10]
[1,10] [15,20]
[1,20]
[1,20] [25,30]
[1,20] [25,35]
[0,20] [25,35]
```

提交文件：Manager.cpp

平台提供的初始文件

main.cpp：

```

#include <iostream>
#include<string>
#include "Manager.hpp"

using namespace std;

int main() {
    IntervalManager<int, 10> int_manager;
    IntervalManager<double, 5> double_manager;

    string type;

    while (cin >> type) {
        if (type == "INT") {
            int s, e;
            cin >> s >> e;
            int_manager.add(s, e);
            int_manager.query();
        } else if (type == "DOUBLE") {
            double s, e;
            cin >> s >> e;
            double_manager.add(s, e);
            double_manager.query();
        }
    }
    return 0;
}

```

Manager.hpp :

```

#ifndef MANAGER_HPP
#define MANAGER_HPP

template <typename T>
struct Segment {
    T start;
    T end;
};

template <typename T, int Limit>
class IntervalManager {
private:
    Segment<T> data[Limit];
    int count;

public:
    IntervalManager();
    void add(T s, T e);
    void query();
};

```

```
extern template class IntervalManager<int, 10>;
extern template class IntervalManager<double, 5>;
#endif
```

周 15-课后 2 过滤器（编程题）

题面

设计类模板 `RangeFilter<typename T, int N>`，每级拦截网含下限 `Low`、上限 `High`、连续容错上限 `Window`、级别 `LevelID`。核心业务：

1. **多级状态隔离**：类内用大小为 `N` 的**静态私有数组** `error_counts[N]`，通过成员函数模板 `isValid` 的非类型模板参数 `LevelID` 精准存取各级错误计数器，防止状态互相污染。
2. **带容错计数（防抖）的过滤判定** `template<int Low, int High, int Window, int LevelID> bool isValid(T val)`：
 - 若 `val` 在安全区间内 (`Low <= val <= High`) → 该级计数器清零，返回 `true`；
 - 若超出区间 → 该级计数器 +1：计数器 $\leq$ `Window` 仍信任，返回 `true`；计数器 $>$ `Window` 拦截，返回 `false`。

输入：每行 `CHECK_INT val` 或 `CHECK_DOUBLE val`。**输出**：每组经双级级联判定（第一级 `LevelID=0`，第二级 `LevelID=1`，第一级 `false` 则短路）。被第一级拦截输出 `DANGER`；过一级被二级拦截输出 `NORMAL`；双过输出 `PERFECT`。

样例输入：
`CHECK_INT 105`
`CHECK_INT 105`
`CHECK_INT 50`
样例输出：
`PERFECT`
`DANGER`
`PERFECT`

提交文件：Filter.cpp

平台提供的初始文件

main.cpp：

```
#include "Filter.hpp"
#include <iostream>
#include <string>

using namespace std;

int main() {
```

```

RangeFilter<int, 2> int_filter;
RangeFilter<double, 2> double_filter;

string op;
while (cin >> op) {
    if (op == "CHECK_INT") {
        int val; cin >> val;
        if (!int_filter.isValid<10, 100, 1, 0>(val)) {
            cout << "DANGER" << endl;
        } else if (!int_filter.isValid<40, 70, 2, 1>(val)) {
            cout << "NORMAL" << endl;
        } else {
            cout << "PERFECT" << endl;
        }
    } else if (op == "CHECK_DOUBLE") {
        double val; cin >> val;
        if (!double_filter.isValid<10, 100, 1, 0>(val)) {
            cout << "DANGER" << endl;
        } else if (!double_filter.isValid<40, 70, 2, 1>(val)) {
            cout << "NORMAL" << endl;
        } else {
            cout << "PERFECT" << endl;
        }
    }
}
return 0;
}

```

Filter.hpp :

```

#ifndef FILTER_HPP
#define FILTER_HPP

template<typename T, int N>
class RangeFilter {
private:
    int error_counts[N];
public:
    RangeFilter();

    template<int Low, int High, int Window, int LevelID>
    bool isValid(T val);
};

extern template class RangeFilter<int, 2>;
extern template class RangeFilter<double, 2>;
extern template bool RangeFilter<int, 2>::isValid<10, 100, 1, 0>(int val);
extern template bool RangeFilter<int, 2>::isValid<40, 70, 2, 1>(int val);
extern template bool RangeFilter<double, 2>::isValid<10, 100, 1, 0>(double val);

```

```
extern template bool RangeFilter<double, 2>::isValid<40, 70, 2, 1>(double val);
#endif
```

周 15-课后 3 三维张量边缘剥离计算（编程题）

题面

编写函数模板 `template<typename T, int D1, int D2, int D3> void AnalyzeTensorShell(T (&tensor)[D1][D2][D3])`：

1. 利用 C++ 模板数组引用推导机制，自动获取三维度 D1、D2、D3（编译期常量）；
2. 计算所有外表面元素的代数和（外表面：任一坐标 = 0 或 = 对应维度 - 1）；
3. 输出格式：Shape: D1xD2xD3 | Shell Sum = 计算结果。

程序从标准输入读一个整数 `test_case`（1 或 2），运行对应内置测试用例。

样例输入 1: 1

样例输出 1: Shape: 2x3x4 | Shell Sum = 20

样例输入 2: 2

样例输出 2: Shape: 3x3x3 | Shell Sum = 26

提交文件：TensorAnalyzer.cpp

平台提供的初始文件

main.cpp：

```
#include "TensorAnalyzer.h"
```

```
int main() {
    int test_case;
    cin >> test_case;

    if (test_case == 1) {
        // 测试用例1：2x3x4 空心立方体
        int cube1[2][3][4] = {
            {{1, 1, 1, 1}, {1, 0, 0, 1}, {1, 1, 1, 1}},
            {{1, 1, 1, 1}, {1, 0, 0, 1}, {1, 1, 1, 1}}
        };
        AnalyzeTensorShell(cube1);
    } else if (test_case == 2) {
        // 测试用例2：3x3x3 中心为2的立方体
        int cube2[3][3][3] = {
            {{1, 1, 1}, {1, 1, 1}, {1, 1, 1}},
            {{1, 1, 1}, {1, 2, 1}, {1, 1, 1}},
            {{1, 1, 1}, {1, 1, 1}, {1, 1, 1}}
        };
        AnalyzeTensorShell(cube2);
    }
}
```

```
    return 0;
}
```

TensorAnalyzer.h :

```
#ifndef TENSOR_ANALYZER_H
#define TENSOR_ANALYZER_H
```

```
#include <iostream>
using namespace std;
```

```
template<typename T, int D1, int D2, int D3>
void AnalyzeTensorShell(T (&tensor)[D1][D2][D3]);
```

// 显式声明需要实例化的函数模板版本

```
extern template void AnalyzeTensorShell<int, 2, 3, 4>(int (&)[2][3][4]);
```

```
extern template void AnalyzeTensorShell<int, 3, 3, 3>(int (&)[3][3][3]);
```

```
#endif
```

周 15-课后 4 变色龙分流打印机（编程题）

题面

不引入右值引用及任何转发机制，纯用**基础主模板（常引用）、函数模板全特化、数组引用重载**实现多分支数据分流，实现以下 SuperPrint：

1. 基础兜底主模板：处理所有未匹配类型，输出 [UNKNOWN_TYPE]；
2. int 全特化：[INT] -> 数值；
3. double 全特化：[DOUBLE] -> 数值；
4. 通用一维数组重载（非字符类型）：[ARRAY] Len=N -> {元素 1,元素 2,...}，逗号分隔无多余逗号；
5. C 风格字符串重载（char 数组）：[C_STRING] Len=N -> 字符串内容（长度含末尾 \0）。

程序从标准输入读 test_case（1 或 2）运行内置用例。

样例输入 1: 1

样例输出 1:

[INT] -> 100

[DOUBLE] -> 3.14

[ARRAY] Len=3 -> {1.5,2.5,3.5}

[C_STRING] Len=8 -> OJ_TEST

[UNKNOWN_TYPE]

样例输入 2: 2

样例输出 2:

[INT] -> -256

[DOUBLE] -> 2.71828
[ARRAY] Len=5 -> {10,20,30,40,50}
[C_STRING] Len=10 -> Hello_C++
[UNKNOWN_TYPE]

提交文件：SuperPrint.cpp

平台提供的初始文件

main.cpp：

```
#include "SuperPrint.h"

int main() {
    int test_case;
    cin >> test_case;

    if (test_case == 1) {
        // 测试用例1
        SuperPrint(100);
        SuperPrint(3.14);
        double scores[3] = {1.5, 2.5, 3.5};
        SuperPrint(scores);
        char path[] = "OJ_TEST";
        SuperPrint(path);
        Dummy d;
        SuperPrint(d);
    } else if (test_case == 2) {
        // 测试用例2
        SuperPrint(-256);
        SuperPrint(2.71828);
        int nums[5] = {10, 20, 30, 40, 50};
        SuperPrint(nums);
        char msg[] = "Hello_C++";
        SuperPrint(msg);
        Test t;
        SuperPrint(t);
    }

    return 0;
}
```

SuperPrint.h：

```
#ifndef SUPER_PRINT_H
#define SUPER_PRINT_H

#include <iostream>
using namespace std;
```

```
// 测试用空结构体定义
```

```

struct Dummy {};
struct Test {};

// 1. 基础兜底主模板 (常引用)
template<typename T>
void SuperPrint(const T& x);

// 2. int 类型全特化声明
template<>
void SuperPrint(const int& x);

// 3. double 类型全特化声明
template<>
void SuperPrint(const double& x);

// 4. 通用一维数组重载声明
template<typename T, int N>
void SuperPrint(T (&arr)[N]);

// 5. C 风格字符数组专属重载声明
template<int N>
void SuperPrint(char (&arr)[N]);

// 显式声明需要实例化的函数模板版本
extern template void SuperPrint<int>(const int&);
extern template void SuperPrint<double>(const double&);
extern template void SuperPrint<double, 3>(double (&)[3]);
extern template void SuperPrint<int, 5>(int (&)[5]);
extern template void SuperPrint<8>(char (&)[8]);
extern template void SuperPrint<10>(char (&)[10]);
extern template void SuperPrint<Dummy>(const Dummy&);
extern template void SuperPrint<Test>(const Test&);

#endif

```

周 15-课堂 1 比特串类型解析 (编程题)

题面

实现 `template<typename T> T decode_bits(const string& bits)`，按目标类型还原数值：

- 32 位类型 (`int/float`) 接收 32 位比特串；64 位类型 (`double`) 接收 64 位比特串。
- 浮点遵循 IEEE 754，整数为有符号整数。
- 大端序**：bits[0] 是编码的最高位比特，bits[1] 次高位，依此类推（比特串索引顺序与编码位地址顺序相反）。
- 因分离编译，需为 `main.cpp` 用到的实例提供**显式实例化**：


```
using namespace std;

template<typename T>
T decode_bits(const string& bits);

extern template int decode_bits<int>(const string&);
extern template float decode_bits<float>(const string&);
extern template double decode_bits<double>(const string&);

#endif
```

周 15-课堂 2 实时动态去重排行榜 Top-K（编程题）

题面

实现 ScoreWall<int K> 维护前 K 名最高分：

1. 同一玩家多次提交：新分 $\leq$ 历史最高 $\rightarrow$ 忽略；新分 $>$ 历史最高 $\rightarrow$ 原地更新并降序重排。
2. 新玩家提交：榜未满 $\rightarrow$ 插入并降序；榜已满 $\rightarrow$ 与最后一名比较，胜出则替换，否则忽略。

不断读入玩家姓名（不含空格）和分数（整数），**每次提交后立即输出完整榜单**，格式 姓名:分数，单空格分隔，行尾无多余空格。

样例输入 1:

Alice 90
Bob 85
Charlie 95
Alice 92
David 88
Eve 91

样例输出 1:

Alice:90
Alice:90 Bob:85
Charlie:95 Alice:90 Bob:85
Charlie:95 Alice:92 Bob:85
Charlie:95 Alice:92 David:88
Charlie:95 Alice:92 Eve:91

样例输入 2:

Tom 80
Jerry 90
Tom 95
Spike 85
Jerry 92
Tyke 88

样例输出 2:

Tom:80
Jerry:90 Tom:80
Tom:95 Jerry:90
Tom:95 Jerry:90 Spike:85
Tom:95 Jerry:92 Spike:85
Tom:95 Jerry:92 Tyke:88

提交文件：ScoreWall.cpp

平台提供的初始文件

ScoreWall.h：

```
#ifndef SCORE_WALL_H
#define SCORE_WALL_H

#include <iostream>
#include <string>
using namespace std;

struct Record {
    string name;
    int score;
};

template<int K>
class ScoreWall {
private:
    Record data[K];
    int count;
public:
    ScoreWall();
    void submit(string name, int score);
    void show();
};

// 显式声明需要实例化的模板版本（平台固定使用K=3）
extern template class ScoreWall<3>;

#endif
```

main.cpp：

```
#include "ScoreWall.h"

int main() {
    ScoreWall<3> wall; // 固定维护前三名
    string name;
    int score;
    while (cin >> name >> score) {
        wall.submit(name, score);
    }
}
```

```
        wall.show();
    }
    return 0;
}
```

周 15-理论题（选择题，20 题）

第 1 题 关于泛型编程与模板，错误的是？ A. 泛型编程独立于特定数据类型 B. 模板实例化由编译器在编译期完成 C. 泛型编程利用模板实现动态多态（运行时多态） D. 模板代码被实例化后才生成机器代码 **答案：C**（模板是静态/编译时多态）

第 2 题 `template <typename T> T foo(T a, T b);` 哪个调用编译错误？ A. `foo(1, 2)` B. `foo(1.5, 2.5)` C. `foo(1, 2.5)` D. `foo<double>(1, 2.5)` **答案：C**（T 推导冲突：int vs double）

第 3 题 关于函数模板特化，正确的是？ A. 特化形式为 `template <typename T>` 后跟特化定义 B. 特化版本逻辑必须与主模板一致 C. 全特化时 `template <>` 尖括号内不保留模板形参 D. 编译器总优先匹配基础模板 **答案：C**

第 4 题 `template<typename T> void print(T){cout<<1;} template<> void print(int){cout<<2;} void print(int){cout<<3;} print(10);` A. 1 B. 2 C. 3 D. 编译错误 **答案：C**（普通非模板函数优先级最高）

第 5 题 类模板成员函数外部实现正确语法？ A. `void Stack<T>::push(T x){}` B. `template <class T> void Stack::push(T x){}` C. `template <typename T> void Stack<T>::push(T x){}` D. `template <typename T> void Stack::push<T>(T x){}` **答案：C**

第 6 题 关于非类型模板形参，正确的是？ A. 任何类对象（如 string）都可作非类型形参 B. 非类型实参必须是编译期常量或常量表达式 C. 类内可当普通变量动态修改 D. C++11 中浮点可无条件作非类型形参 **答案：B**

第 7 题 `template <typename T, int N> class Array;` 哪组类型完全相同？ A. `Array<int,5>` 与 `Array<int,10>` B. `Array<int,5>` 与 `Array<double,5>` C. `Array<int,2+3>` 与 `Array<int,5>` D. `Array<const int,5>` 与 `Array<int,5>` **答案：C**

第 8 题 普通非模板函数与同名函数模板，完美匹配时重载决议？ A. 优先选非模板函数 B. 优先选模板基础版本 C. 优先选模板特化版本 D. 报歧义错误 **答案：A**

第 9 题 `template <typename T, int N> void getLength(T (&arr)[N]); int a[10]; getLength(a);` 推导？ A. T 是 `int*`，N=10 B. T 是 `int`，N=10 C. T 是 `int[10]`，N 无法推导 D. 编译报错 **答案：B**

第 10 题 若改为 `void getLength(T arr[N])`，`getLength(a);`？ A. 仍输出 10 B. 能推导 N 不能推导 T C. 编译错误，arr 退化为指针无法推导 N D. 段错误 **答案：C**

第 11 题 关于类模板全特化，错误的是？ A. 只有类模板可全特化，函数模板只能重载不能特化 B. 全特化时类名后尖括号显式指定类型 C. 全特化版本可定义完全不同的成员 D. 特化前要写空模板参数列表 `template <>` **答案：A**（函数模板也可全特化）

第 12 题 `template <int Size> class Buffer{int data[Size];}; cin>>n; Buffer<n> myBuf;` A. 编译成功运行期动态分配 B. 编译失败，n 是运行期变量非编译期常量 C. 编译成功但越界 D. 输入正整数才能编译成功 **答案：B**

第 13 题 关于模板头文件包含模式，最准确的是？ A. 声明实现分别放 .h 和 .cpp 独立编译 B. 模板声明与实现通常都写在同一 .h 中 C. .cpp 只含声明不需实现就能链接 D. 未被实例化的模板源文件会生成完整目标代码 **答案：B**

第 14 题 `template <typename T, int R, int C> void processMatrix(T (&m)[R][C]); double mat[3][5]; processMatrix(mat);` A. R=5,C=3 B. R=3,C=5 C. R=15,C 无法推导 D. 编译错误 **答案：B**

第 15 题 C++11 下哪种类型不能作非类型模板形参？ A. char B. long long C. double D. 指针类型 **答案：C**

第 16 题 `template<typename T> void swap(T& a,T& b);` 显式实例化正确语法？ A. `template swap<int>(int&,int&);` B. `template void swap<int>(int&,int&);` C. `void swap<int>(int&,int&);` D. `template<> void swap<int>(int&,int&);` **答案：B**

第 17 题 `template<typename T> void foo(T){cout<<"A";} template<typename T> void foo(T*){cout<<"B";} template<> void foo(int){cout<<"C";} foo(5);` A. A B. B C. C D. 歧义 **答案：C**（5 推导为 int，命中全特化）

第 18 题 接上题，`int n=10; foo(&n);` A. A B. B C. C D. 编译错误 **答案：B**（&n 是 int，命中 `foo(T)` 偏序更精确）

第 19 题 关于类模板内 `typename`，正确的是？ A. 与 `class` 任何场景可互换 B. 声明嵌套从属依赖类型时必须前置 `typename` C. 只能在非类型模板参数声明中用 D. 实例化模板对象时必须写 `typename` **答案：B**

第 20 题 `template <typename T, typename U> class Pair;` 想为 T、U 相同时编写专门逻辑（不用特化），可？ A. 无法处理，必须偏特化 B. 成员函数内用 `if constexpr (std::is_same_v<T,U>)` 编译期分支 C. 写成 `template <typename T, typename T>` D. 构造函数中强转 U 为 T **答案：B**

说明：本文件由 Matrix 平台题目数据整理，含完整题面、提交文件、平台初始文件；选择题答案为平台标准答案。